

SCALE FOR PROJECT PYTHON MODULE (/PROJECTS/PYTHON-MODULE-10)

You should evaluate 1 student in this team



Git repository

git@vogsphere.42porto.com:vogsphere/intra-uuid-514bc2ec-bcbb-4t

Introduction

- Remain polite, courteous, respectful and constructive throughout the evaluation process. The well-being of the community depends on it.
- Identify with the person (or the group) evaluated the eventual dysfunctions of the work. Take the time to discuss and debate the problems you have identified.
- You must consider that there might be some difference in how your peers might have understood the project's instructions and the scope of its functionalities. Always keep an open mind and grade him/her as honestly as possible. The pedagogy is valid only and only if peer-evaluation is conducted seriously.

Guidelines

- Only grade the work that is in the student or group's GiT repository.
- Double-check that the GiT repository belongs to the student or the group. Ensure that the work is for the relevant project and also check that "git clone" is used in an empty folder.
- Check carefully that no malicious aliases was used to fool you and make you evaluate something other than the content of the official repository.
- To avoid any surprises, carefully check that both the evaluating and the evaluated students have reviewed the possible scripts used to facilitate the grading.
- If the evaluating student has not completed that particular project yet, it is mandatory for this student to read the entire subject prior to starting the defence.
- Use the flags available on this scale to signal an empty repository, non-functioning program, a norm error, cheating etc. In these cases, the grading is over and the final grade is 0 (or -42 in case of cheating). However, with the exception of cheating, you are encouraged to continue to discuss your work (even if you have not finished it) in order to identify any issues that may have caused this failure and avoid repeating the same mistake in the future.
- Remember that for the duration of the defence, no segfault, no other unexpected, premature, uncontrolled or unexpected termination of the program, else the final grade is 0. Use the appropriate flag.
You should never have to edit any file except the configuration file if it exists.
If you want to edit a file, take the time to explicit the reasons with the evaluated student and make sure both of you are okay with this.
- You must also verify the absence of memory leaks. Any memory allocated on the heap must

be properly freed before the end of execution.

You are allowed to use any of the different tools available on the computer, such as leaks, valgrind, or e_fence. In case of memory leaks, tick the appropriate flag.

Attachments

 [subject.pdf](https://cdn.intra.42.fr/pdf/pdf/196511/en.subject.pdf) (https://cdn.intra.42.fr/pdf/pdf/196511/en.subject.pdf)

 [generator.tar.gz](https://cdn.intra.42.fr/document/document/45481/generator.tar.gz) (https://cdn.intra.42.fr/document/document/45481/generator.tar.gz)

Preliminaries

Basics

- Only grade the work that is in the learner's or group's GiT repository.
- Double-check that the GiT repository belongs to the learner or the group. Ensure that the work is for the relevant activity and also check that "git clone" is used in an empty folder.
- Check carefully that no malicious aliases were used to fool you and make you evaluate something other than the content of the official repository.
- To avoid any surprises, carefully check that both the evaluating and the evaluated learners have reviewed the possible scripts used to facilitate the grading.
- If the evaluating learner has not completed that particular activity yet, it is mandatory for this learner to read the entire subject prior to starting the defence.
- Remember that for the duration of the defence, no segfault, no other unexpected, premature, uncontrolled, or unexpected termination of the program, else the final grade is 0.
You should never have to edit any file except the configuration file, if it exists.
If you want to edit a file, take the time to explicit the reasons with the evaluated learner and make sure both of you are okay with this.
- Check that only the requested files are available in the git repository.
If not, the peer-review stops here.

 Yes

 No

Exercises

Exercise 0 - Lambda Sanctum

Check that lambda_spells.py is properly implemented:

- Does artifact_sorter() use sorted() with a lambda to sort by 'power' descending?
- Does power_filter() use filter() with a lambda to filter mages by minimum power?
- Does spell_transformer() use map() with a lambda to add * prefix and suffix?
- Does mage_stats() use lambdas with max(), min(), and sum() for statistics?

Test the functions with sample data:

- Create test artifacts and verify sorting works correctly
- Test power filtering with different minimum values
- Verify spell transformation adds the correct decorations
- Check that statistics calculation returns correct max, min, and average

Verify that lambda expressions are used appropriately (not def functions for simple operations).
If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

 Yes

 No

Exercise 1 - Higher Realm

Check that higher_magic.py demonstrates higher-order functions:

- Does spell_combiner() return a function that calls both input functions?

- Does `power_amplifier()` return a function that multiplies the base spell result?
- Does `conditional_caster()` return a function that checks condition before casting?
- Does `spell_sequence()` return a function that executes all spells in order?

Test the higher-order function behavior:

- Create simple test functions and verify combination works
- Test power amplification with different multipliers
- Verify conditional casting works with both true and false conditions
- Check that spell sequences execute all spells and return results list

Verify that functions are treated as first-class citizens (passed as arguments, returned from function). If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

☒ Yes

☐ No

Exercise 2 - Memory Depths

Check that `scope_mysteries.py` demonstrates lexical scoping and closures:

- Does `mage_counter()` return a function that maintains count state?
- Does `spell_accumulator()` return a function that accumulates power over calls?
- Does `enchantment_factory()` return functions that remember their enchantment type?
- Does `memory_vault()` return a dict with store/recall functions sharing private storage?

Test closure behavior:

- Create multiple counters and verify they maintain separate state
- Test power accumulation with different initial values
- Create different enchantment factories and verify they work independently
- Test memory vault storage and recall functionality

Verify that closures properly capture and maintain their lexical environment.

If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

☒ Yes

☐ No

Exercise 3 - Ancient Library

Check that `functools_artifacts.py` properly uses the `functools` module:

- Does `spell_reducer()` use `functools.reduce` with operator module functions?
- Does `partial_enchanter()` use `functools.partial` to create specialized functions?
- Does `memoized_fibonacci()` use `functools.lru_cache` for memoization?
- Does `spell_dispatcher()` use `functools singledispatch` for type-based dispatch?

Test `functools` functionality:

- Test `reduce` with different operations (add, multiply, max, min)
- Verify partial application creates working specialized functions
- Test `fibonacci` memoization improves performance on repeated calls
- Check single dispatch handles different types appropriately

Verify that `functools` features are used correctly, not manually implemented.

If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

☒ Yes

☐ No

Exercise 4 - Master's Tower

Check that `decorator_mastery.py` demonstrates decorator mastery:

- Does `spell_timer()` create a decorator that measures execution time?
- Does `power_validator()` create a parameterized decorator for validation?
- Does `retry_spell()` create a decorator that retries failed functions?
- Does `MageGuild` class properly use `@staticmethod`?

Test decorator functionality:

- Test `spell_timer` with a function and verify timing output
- Test `power_validator` with valid and invalid power levels
- Test `retry_spell` with functions that fail and eventually succeed
- Verify `staticmethod` works without instance and instance method works with validation

Check that `functools.wraps` is used to preserve function metadata.

Verify that decorators properly transform function behavior.

If everything works correctly, tick "Yes." Otherwise, tick "No" and the peer-review is over.

✓ Yes

✗ No

Code Quality and Understanding

Code Quality and Understanding

Evaluate the learner's understanding of functional programming concepts:

- Can the learner explain the difference between lambda and def functions?
- Can the learner explain what makes functions "first-class citizens"?
- Does the learner understand how closures capture lexical scope?
- Can the learner explain the benefits of functools utilities?
- Does the learner understand how decorators transform functions?

Check for code quality:

- Is the code written in Python 3.10 or higher?
- Does the code adhere to the flake8 linter standards (no errors)?
- Are type hints REQUIRED for all functions and methods (parameters and return values)?
- Docstrings are NOT required for this module.
- Is the code clean and readable?
- Are functional programming patterns used correctly?
- Does the code demonstrate understanding rather than just copying?

Discuss the concepts with the learner to assess their understanding.
If the learner demonstrates good understanding of functional programming, tick "Yes."

✓ Yes

✗ No

Ratings

Don't forget to check the flag corresponding to the defense

✓ Ok

★ Outstanding project

Empty work

📄 Incomplete work

⚙️ Invalid compilation

📋 Norme

📄 Cheat

⚠️ Concerning situation

💧 Leaks

🚫 Forbidden function

💬 Can't support /

Conclusion

Leave a comment on this evaluation (2048 chars max)

Finish evaluation